Name: Gunjan Chandnani Roll No : 13 Div : D20A

Experiment 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

What is a Smart Contract?

A Smart Contract is a self-executing digital agreement stored on a blockchain. The terms and conditions of the agreement are written in computer code. When the predefined conditions are met, the contract automatically executes the action without requiring a middleman.

Smart contracts run on blockchain networks and are transparent, secure, and immutable. Once deployed, the code cannot be easily modified, which ensures trust among participants.

Example:
In a supply chain system, when a product is delivered, the payment can automatically be released to the manufacturer through a smart contract.

Key Features of Smart Contracts:

- Automation: Executes automatically when conditions are satisfied.
- Transparency: All participants can see the transaction details on the blockchain.
- Security: Data stored in the blockchain cannot be easily changed.
- Trust: Eliminates the need for third-party intermediaries.

Significance of Smart Contracts in Ethereum Blockchain

Smart contracts play a major role in the Ethereum blockchain ecosystem. Ethereum allows developers to create decentralized applications (DApps) that use smart contracts to automate processes.

Importance of Smart Contracts in Ethereum:

1. Automation of Transactions
   Smart contracts automatically execute transactions when specific conditions are met, reducing manual work.
2. Elimination of Intermediaries
   They remove the need for middlemen such as banks, brokers, or third parties, reducing cost and time.
3. Transparency
   All transactions and contract activities are recorded on the blockchain and can be verified by participants.
4. Security
   Smart contracts use cryptographic techniques that make the system highly secure and resistant to fraud.
5. Reliability
   Once deployed on the blockchain, smart contracts run exactly as programmed without downtime

Example in Supply Chain:
In a blockchain-based supply chain system, smart contracts can automatically update product status such as Manufactured, Shipped, and Delivered. This improves tracking, transparency, and trust among manufacturers, distributors, and retailers.

Steps:

Step 1: Open Remix IDE

1. Open your browser (Chrome).
2. Go to **https://remix.ethereum.org**
3. The **Remix IDE** interface will open.

Step 2: Create a New Solidity File

1. On the **left side panel**, click the **file icon**.
2. Click **Create New File**.
3. Name the file: SupplyChain.sol

Step 3: Write the Smart Contract Code

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract SupplyChain {

    struct Product {

        uint id;

        string name;

        string manufacturer;

        string status;

    }

    Product public product;

    // Create Product

    function createProduct(uint _id, string memory _name, string memory _manufacturer) public {

        product = Product(_id, _name, _manufacturer, "Manufactured");

    }
```

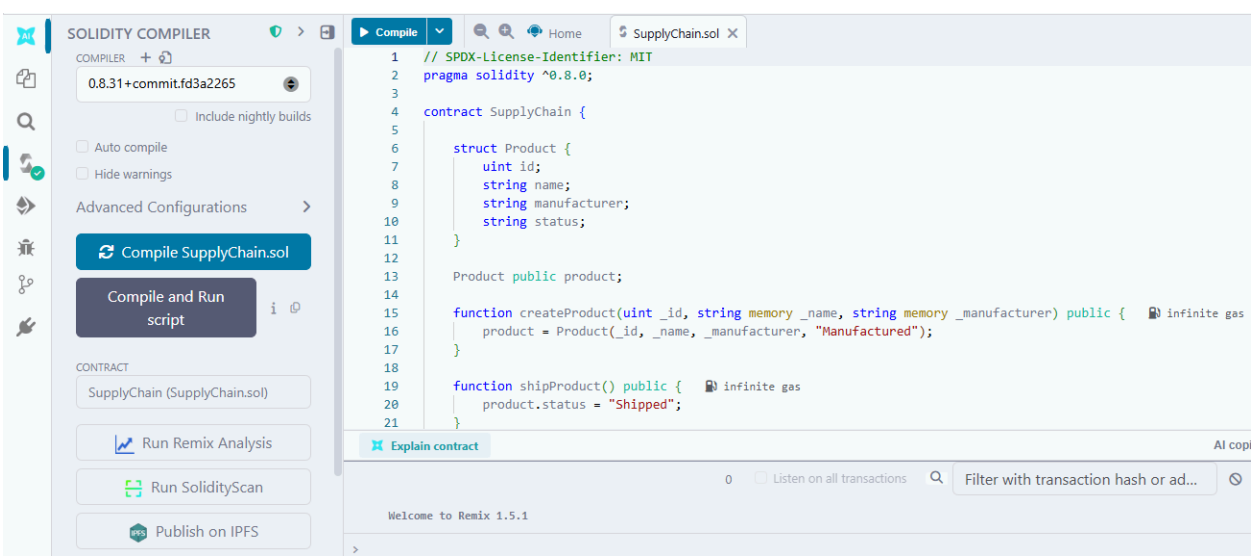
```
    // Get Product Details
    function getProduct() public view returns(uint, string memory, string memory, string memory) {
        return (product.id, product.name, product.manufacturer, product.status);
    }
    // Update Product Status
    function updateProduct(string memory _status) public {
        product.status = _status;
    }
}
```

This smart contract will:

- createProduct
- getProduct
- updateProduct
- product

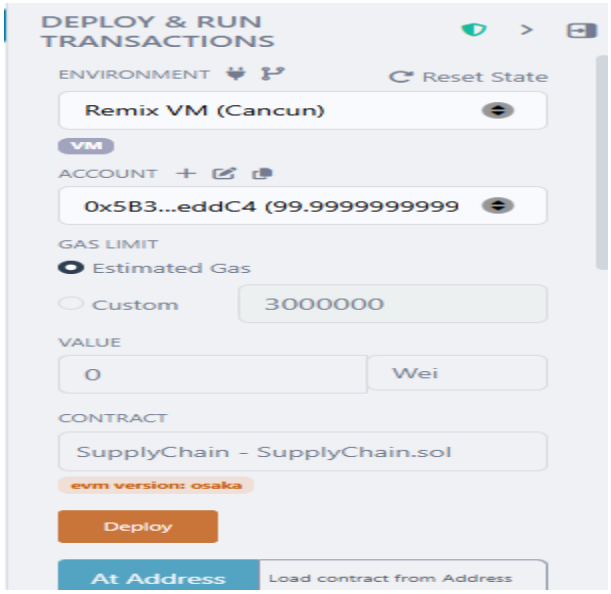
Step 4: Compile the Smart Contract

1. On the **left side**, click the **Solidity Compiler icon**.
2. Select compiler version **0.8.0 or above**.
3. Click **Compile SupplyChain.sol**.
   If there is **no error**, compilation is successful.



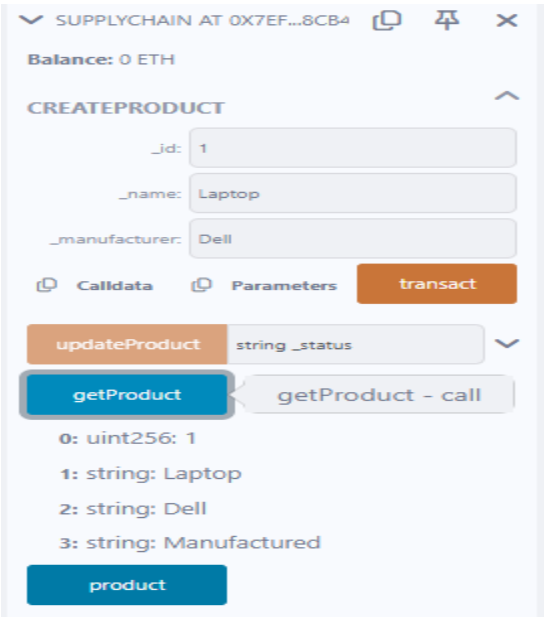
Step 5: Deploy the Smart Contract

1. Click **Deploy & Run Transactions** (Ethereum icon).
2. In **Environment**, select: Remix VM (Cancun)



Step 6: Test the Contract (Perform Transactions)

Click **createProduct**



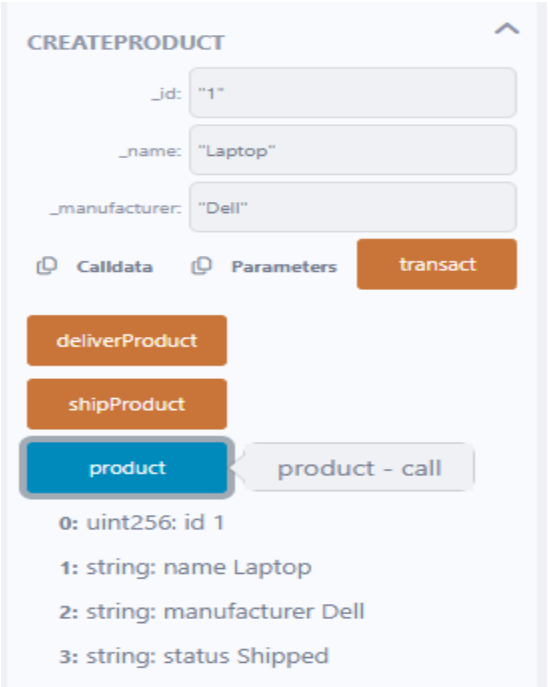
Update Product

Enter:

Shipped

Click **Transact**.

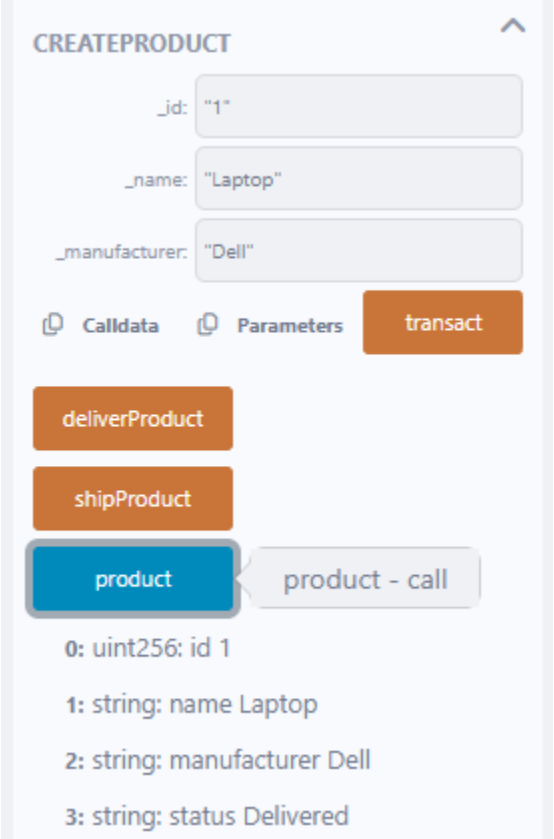
Then run **getProduct** again.



Enter: Delivered

Click **Transact**.

Then run **getProduct** again.



Conclusion

In this experiment, the concept of smart contracts and their importance in the Ethereum blockchain were studied. Smart contracts enable automatic execution of agreements without the need for intermediaries, making transactions faster, transparent, and secure. By using blockchain technology, all contract activities are recorded in a decentralized and immutable ledger, which increases trust among participants. Overall, smart contracts play a significant role in building decentralized applications and improving systems such as supply chain management by ensuring transparency, efficiency, and reliability.